

Day 3

Overview

Dashboard

Dimensionality Reduction
(PCA)

Clustering (K-Means)

Neural Networks

← Back to Day 3

Clustering: K-Means

Unsupervised learning to group similar data points together.

K-Means Algorithm

K-means is an iterative algorithm that partitions a dataset into K distinct, non-overlapping subgroups (clusters) where each data point belongs to the cluster with the nearest mean.

Objective Function

The goal is to minimize the within-cluster sum of squares (variance):

$$\min_{C, \{\mu_k\}} \sum_{k=1}^K \sum_{i: C(i)=k} \|x_i - \mu_k\|^2$$

Where:

- $C(i)$ is the cluster assignment for data point x_i .
- μ_k is the centroid (mean) of cluster k .

Algorithm Steps

1. **Initialize:** Choose K initial centroids randomly.
2. **Assignment:** Assign each point to the nearest centroid.
3. **Update:** Recompute the centroid of each cluster (mean of assigned points).
4. **Repeat:** Repeat steps 2 and 3 until a stopping condition is met:
 - **Convergence:** Centroids do not move significantly.
 - **No Change:** Cluster assignments remain the same.
 - **Max Iterations:** A predefined limit of iterations is reached.

Interactive K-Means Demo

Click on the canvas to add data points. Then click **"Run Step"** to see K-Means in action (K=3).

Day 3

Overview

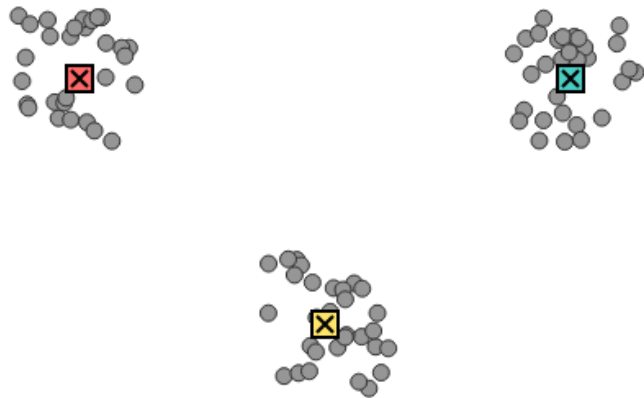
Dashboard

Dimensionality Reduction
(PCA)

Clustering (K-Means)

Neural Networks

← Back to Day 3



Generated 3 clusters with initialized centroids. Click 'Run Step' to assign.

Python Implementation (scikit-learn)

Here is how to implement K-Means clustering using scikit-learn, along with other clustering methods like GMM and Spectral Clustering.

```
from sklearn.cluster import KMeans, DBSCAN, SpectralClustering
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler
from sklearn import datasets
import matplotlib.pyplot as plt

def clustering_demo():
    # Generate synthetic data (two moons)
    X, y = datasets.make_moons(n_samples=400, noise=0.08,
                               random_state=0)
    scaler = StandardScaler()
    X_s = scaler.fit_transform(X)

    # KMeans
    kmeans = KMeans(n_clusters=2, random_state=0)
    labels_km = kmeans.fit_predict(X_s)

    # Gaussian Mixture Models (GMM)
    gmm = GaussianMixture(n_components=2, random_state=0)
    labels_gmm = gmm.fit_predict(X_s)
```

Day 3

Overview

Dashboard

Dimensionality Reduction
(PCA)

Clustering (K-Means)

Neural Networks

← Back to Day 3

```
# Plotting (helper function)
plt.scatter(X_s[:, 0], X_s[:, 1], c=labels_km,
            cmap='viridis')
plt.title("KMeans on two moons")
plt.show()
clustering_demo()
```